

DATA 260 Homework 1 Report

Student: Xuanhua Li

Student ID: 019122974

Assigned Domain: Rental Housing Listings

Hardware: Apple MacBook Air, M4, 16 GB memory

Local Model: qwen3:8b

Code/Results Commit:

e41e708ad1bce356dae97d8bda5200aea2cb9ba1

Submission Tag: hw1

GitHub Repository:

<https://github.com/carrovo/data260-2974>

SID4	2974
PORT_BASE	8274
PREFIX	s2974
SEED	2974
VERIFY_SEED	262974
DOMAIN_ID	6

Part1

I. HTML

1. Create an HTML page titled HW1-{Your Name}.
2. Add a heading tag naming your assigned domain entity (use the header tag that renders the biggest font size).

```
code > web_application > <> index.html > html > head
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7      <title>HW1-Xuanhua Li</title>
8
9      <style>
10         body {
11             font-family: Arial, sans-serif;
12             max-width: 700px;
13             margin: 40px auto;
14             padding: 20px;
15             background-color: #f4f6f8;
16             color: #263238;
17         }
18
19         h1 {
20             text-align: center;
21         }
22
```

3. Before coding, create DOMAIN_SCHEMA.md defining your entity's fields and category Value

```
DOMAIN_SCHEMA.md > # Domain Schema
1  # Domain Schema
2
3  ## Personal Configuration
4
5  - SID4: 2974
6  - PORT_BASE: 8274
7  - PREFIX: s2974
8  - SEED: 2974
9  - VERIFY_SEED: 262974
10 - DOMAIN_ID: 6
11 - Assigned Domain: Rental housing listings
12
13 ## Entity
14
15 Entity name: Rental Listing
16
17 ## Fields
18
19 - listingTitle: Required text. The primary field for the rental listing.
20 - address: Required text. The property address.
21 - submitterEmail: Required email address.
22 - description: Required text area. Must contain more than 25 characters.
23 - propertyType: Required category selected from the dropdown menu.
24 - termsAccepted: Required boolean indicating acceptance of the terms and conditions.
25 - submissionDate: Date and time added after successful form validation.
26
27 ## Property Type Categories
28
29 - Apartment
30 - House
31 - Studio
32 - Shared Room
33
34
```

4. Create a form for your domain entity. The form should include the following:
- a) A required text input for the entity's primary field, with a placeholder describing it. The cursor should automatically focus on this field when the page loads.
 - b) A required text input for the entity's secondary field, with a descriptive placeholder.
 - c) A required email input for the submitter's email, with a descriptive placeholder.
 - d) A required text area for the entity's content/description field, with a descriptive Placeholder.

```
<body>
  <!-- The largest heading identifies the assigned domain. -->
  <h1>Rental Housing Listings</h1>

  <form id="rentalForm">
    <!-- Primary field: required and automatically focused. -->
    <label for="listingTitle">Listing Title</label>
    <input
      type="text"
      id="listingTitle"
      name="listingTitle"
      placeholder="Enter a title for the rental listing"
      required
      autofocus
    >

    <!-- Secondary field. -->
    <label for="address">Property Address</label>
    <input
      type="text"
      id="address"
      name="address"
      placeholder="Enter the rental property address"
      required
    >

    <!-- Submitter email. -->
    <label for="submitterEmail">Submitter Email</label>
    <input
      type="email"
      id="submitterEmail"
      name="submitterEmail"
      placeholder="Enter your email address"
      required
    >

    <!-- Content field. -->
    <label for="description">Property Description</label>
    <textarea
      id="description"
      name="description"
      placeholder="Describe the rental property in more than 25 characters"
      required
    ></textarea>
  </form>
</body>
```

- e) A dropdown for category selection, with four domain-appropriate options.
- f) A checkbox and a label with the text "I agree to the terms and conditions."
- g) A submit button with domain-appropriate text

```

<!-- Four domain-appropriate categories. -->
<label for="propertyType">Property Type</label>
<select id="propertyType" name="propertyType" required>
  <option value="" disabled selected>Select a property type</option>
  <option value="Apartment">Apartment</option>
  <option value="House">House</option>
  <option value="Studio">Studio</option>
  <option value="Shared Room">Shared Room</option>
</select>

<!-- Terms and conditions checkbox. -->
<div class="terms">
  <input
    type="checkbox"
    id="termsAccepted"
    name="termsAccepted"
  >
  <label for="termsAccepted">
    I agree to the terms and conditions.
  </label>
</div>

<button type="submit">Submit Rental Listing</button>
</form>

```

5. Add a script tag to link your JavaScript code for Part II at the end of your HTML file

```

<!-- JavaScript is linked at the end of the HTML body. -->
<script src="feedback.js"></script>
</body>
</html>

```

II. Javascript

1. Write a JavaScript function using an arrow function to validate:

- Verify that the content field has more than 25 characters. If validation fails, display an alert with an appropriate message.
- Verify that the terms and conditions checkbox is checked. If validation fails, display an alert with an appropriate message.

```

// Arrow function: validates the description and terms checkbox.
const validateForm = () => {
  const description = document
    .getElementById("description")
    .value
    .trim();

  const termsAccepted =
    document.getElementById("termsAccepted").checked;

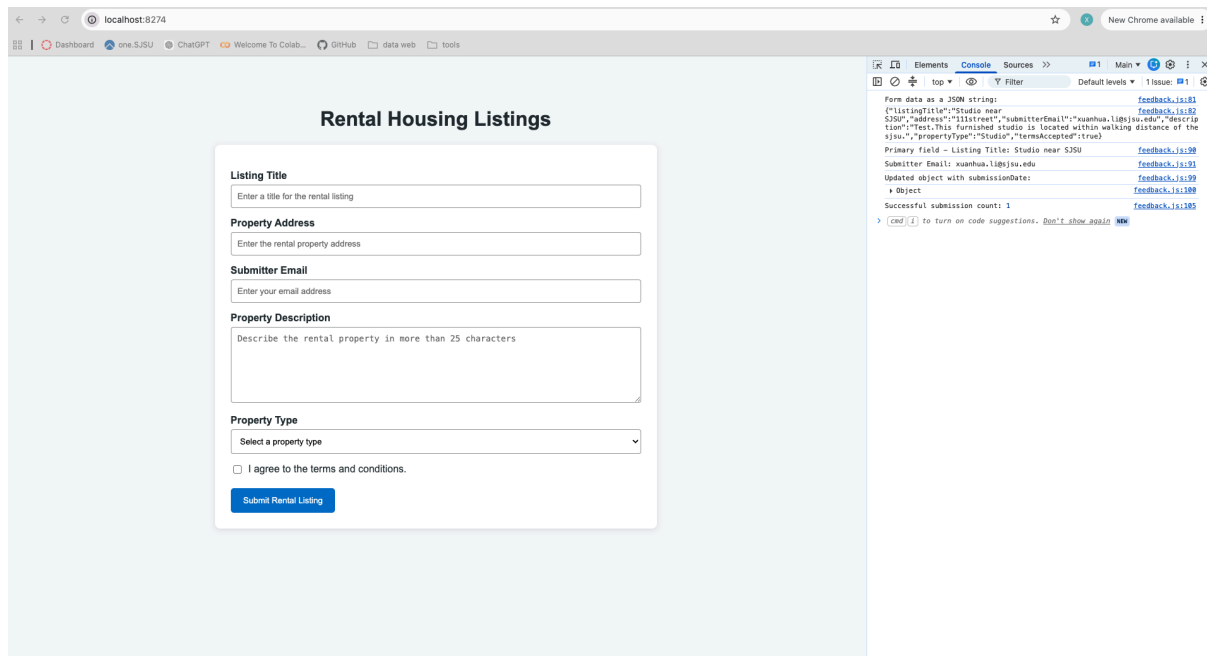
  if (description.length <= 25) {
    alert("The property description must contain more than 25 characters.");
    return false;
  }

  if (!termsAccepted) {
    alert("You must agree to the terms and conditions.");
    return false;
  }

  return true;
};

```

2. After the form submission is successful, convert the form data into a JSON string and log the output in the console.
3. Use object destructuring to extract the primary field and email field from the parsed object and log their values in the console.
4. Use the spread operator to add a new field `submissionDate` with the current date and time to the parsed object. Log the updated parsed object in the console.
5. Create a closure to track how many times the form has been successfully submitted and log the submission count each time the form is submitted

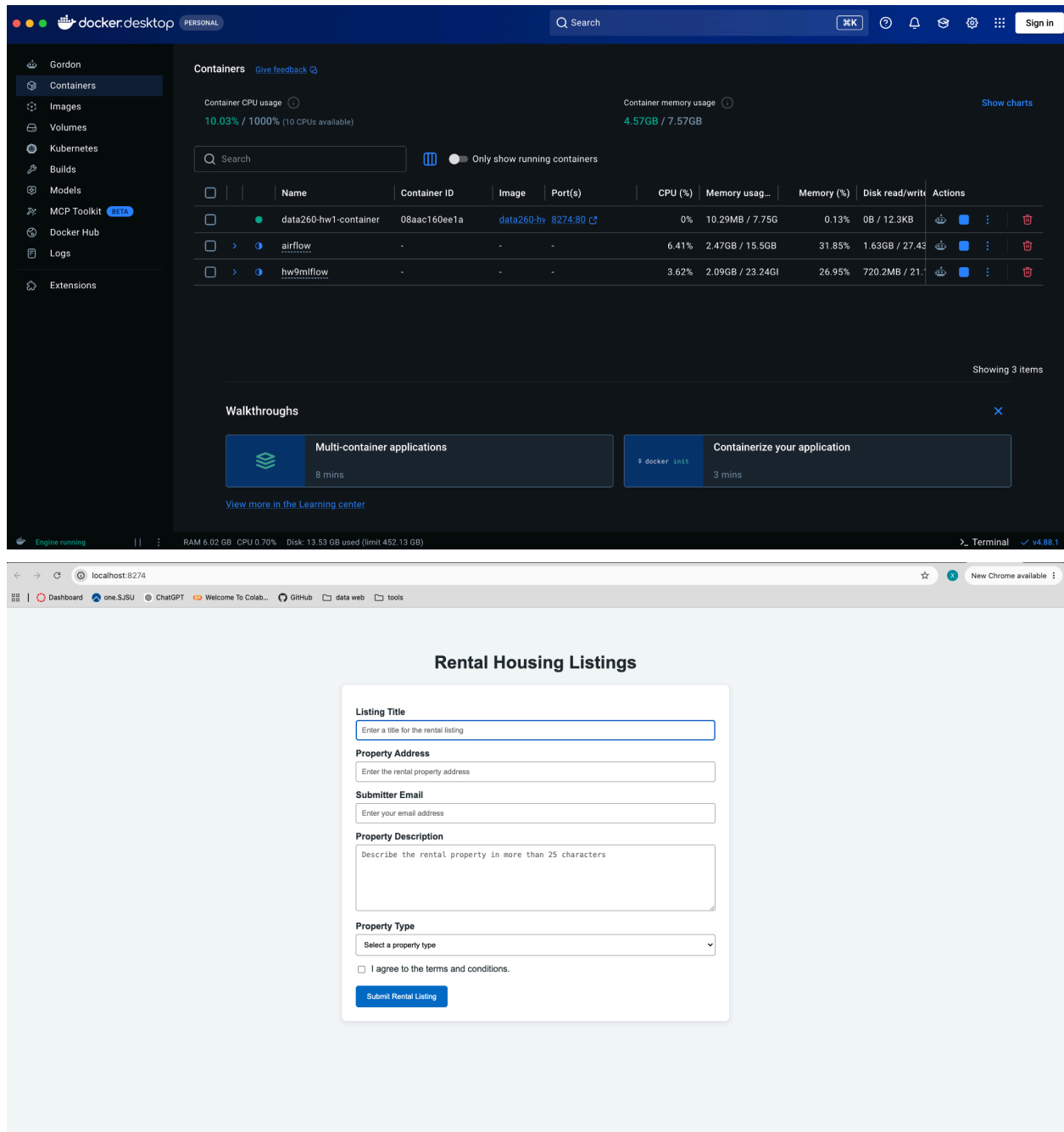


The console output shows the JSON string, destructured listing title and email, the updated object with `submissionDate`, and the closure-based successful submission count.

DEPLOYMENT:

Docker: Create a docker image of the above application and build and run the application using docker in your local.

Provide the screenshot of your app running on your localhost.



The Docker container mapped localhost port 8274 to Nginx port 80, and the application loaded successfully at <http://localhost:8274>.

AWS ECS: Create an AWS ECS service (only one task) running the above application using the docker image created above.

Provide the screenshot of your app running on your public IP address.

I built the web application as a Docker image and pushed the image to a private Amazon Elastic Container Registry repository. I then created an Amazon ECS task definition and deployed one task using AWS Fargate.

Deployment configuration:

- AWS Region: us-east-1
- ECR repository: s2974-hw1-web-app
- ECS cluster: s2974-hw1-cluster
- Task definition family: s2974-hw1-task
- ECS service: s2974-hw1-service
- Launch type: AWS Fargate
- Desired task count: 1
- Container port: 80
- Public IP assignment: Enabled

The ECS task reached the Running state, and I successfully opened the Rental Housing Listings application through the task's public IPv4 address.

After collecting the required evidence, I stopped the running task and deleted the ECS service and cluster to prevent unnecessary AWS charges.

The screenshot shows the AWS Management Console interface for the Amazon Elastic Container Registry (ECR). The main content area displays the details for the repository 's2974-hw1-web-app' in the 'us-east-1' region. The 'Images' tab is selected, showing a list of three images: 'latest', '-', and '-'. Each image entry includes its type (Image Index, Other, or Image), creation time, size, digest, and last pulled time. The 'latest' image is the most recent, created on August 30, 2026, at 19:30:59 UTC-07, with a size of 26.27 MB and a digest of sha256:af008adaafc554b... The other two images are also created on the same date and time, with sizes of 0.00 MB and 26.27 MB respectively, and digests of sha256:fac9148132b2580... and sha256:74ef9f7f0b6c98af... The console also shows the repository's lifecycle policy, permissions, and repository tags. The left sidebar contains navigation links for Amazon ECR, Private registry, Public registry, and various AWS services like ECR public gallery, Amazon ECS, and Amazon EKS.

Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
latest	Image Index	August 30, 2026, 19:30:59 (UTC-07)	26.27	sha256:af008adaafc554b...	-
-	Other	August 30, 2026, 19:30:59 (UTC-07)	0.00	sha256:fac9148132b2580...	-
-	Image	August 30, 2026, 19:30:58 (UTC-07)	26.27	sha256:74ef9f7f0b6c98af...	-

Amazon Elastic Container Service

Favorites and recents

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions

Account settings

AWS Resource Explorer

AWS Batch

Amazon ECR

Repositories

Online learning workshop

Documentation

Discover products

Subscriptions

Tell us what you think

Amazon Elastic Container Service

Clusters > s2974-hw1-cluster > Tasks > 76772cde691c4650aefc3cd837d81dc > Configuration

Run Reachability Analyzer

Networking

Network mode
awsipc

Private IP
172.31.22.228

Task scale-in protection

Protection status
Turned off

ENI ID
eni-Oc13be744fa7aa5e

Public IP
34.228.31.225 | open address

Subnet
subnet-00cc9d9bd419a42f5

IPV6 address
-

Security group
sg-0e8da81af45fec31b (s2974-hw1-sg)

MAC address
0a:ff:c9:ed:3b:49

Configure task scale-in protection

Containers (1)

Filter containers

Container name
s2974-web-container

Container runtime ID
76772cde691c46...

Image URI
99660153...

Image Digest
sha256:af...

Status
Running

Health status
Unknown

CPU
0

Memory hard/soft limit
- / -

Container details for s2974-web-container

Details

Log configuration

Restart policy

Network bindings

Docker labels and hosts

Environment variables and files

Volume configuration

Managed agents

Details

Image URI
skr.ecr.us-east-1.amazonaws.com/s2974-hw1-web-app@sha256:af008adaafc554bd91791832a2b304a0b1ad70105e412d574fa84b859123a647

Essential
Yes

Command
-

Amazon Elastic Container Service

Favorites and recents

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions

Account settings

AWS Resource Explorer

AWS Batch

Amazon ECR

Repositories

Online learning workshop

Documentation

Discover products

Subscriptions

Tell us what you think

Amazon Elastic Container Service

Clusters > s2974-hw1-cluster > Tasks

s2974-hw1-service deployment is in progress. It takes a few minutes.

View in CloudFormation

Notifications

20

3

1

Last updated
August 30, 2026, 19:54 [UTC-7:00]

Actions

Create with Express Mode

s2974-hw1-cluster

Cluster overview

Cluster ARN
arn:aws:ecs:us-east-1:123456789012:cluster/s2974-hw1-cluster

Status
Active

CloudWatch monitoring
Default

Action logs
Turned off

Services
0 active | 0 draining

Tasks
0 running | 0 pending

Registered container instances
-

Services

Tasks

Daemons

Infrastructure

Metrics

Scheduled tasks

Configuration - updated

Event history

Tags

Tasks (1)

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

Manage tags

Stop

Run new task

Task

Last status

Desired status

Task definition

Health status

Started by

Created at

Started at

Group

76772cde691c4650a...

Running

Running

s2974-hw1-task-1

Unknown

ecs-svc/46555296...

2 minutes ago

2 minutes ago

service:s2974-

The screenshot shows a web browser window with the address bar displaying '34.228.31.225'. The browser's tab bar includes 'Dashboard', 'one.SJSU', 'ChatGPT', 'Welcome To Colab...', 'GitHub', 'data web', 'tools', and 'Console Home | C...'. The main content area features a light blue background with a white form titled 'Rental Housing Listings' centered on the page. The form contains the following fields and elements:

- Listing Title**: A text input field with the placeholder text 'Enter a title for the rental listing'.
- Property Address**: A text input field with the placeholder text 'Enter the rental property address'.
- Submitter Email**: A text input field with the placeholder text 'Enter your email address'.
- Property Description**: A text area with the placeholder text 'Describe the rental property in more than 25 characters'.
- Property Type**: A dropdown menu with the placeholder text 'Select a property type'.
- ☐ I agree to the terms and conditions.
- Submit Rental Listing**: A blue button.

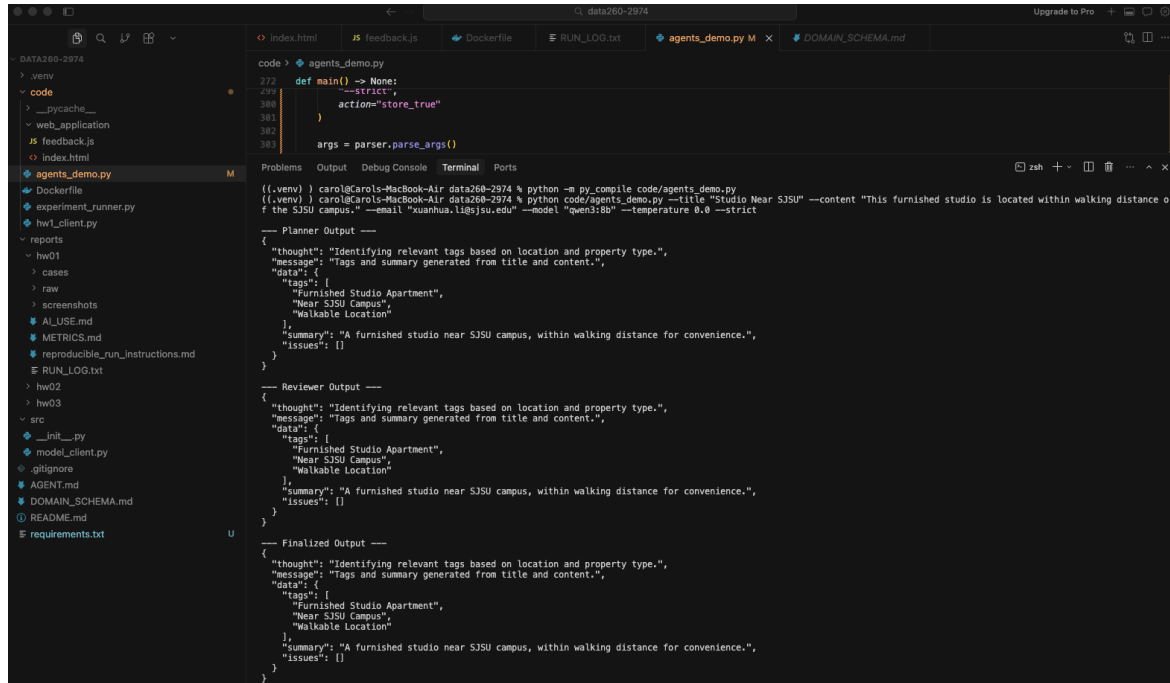
The Docker image was uploaded to Amazon ECR and deployed as one AWS Fargate task. The task reached RUNNING status, received a public IPv4 address, and successfully served the application over HTTP port 80.

Agentic AI Part2

1. Command used (the exact python agents_demo.py line).

python code/agents_demo.py --title "Studio Near SJSU" --content "This furnished studio is located within walking distance of the SJSU campus." --email "xuanhua.li@sjsu.edu" --model "qwen3:8b" --temperature 0.0 --strict

2. Console screenshot(s) showing: Planner output, Reviewer output, Finalized outputs.



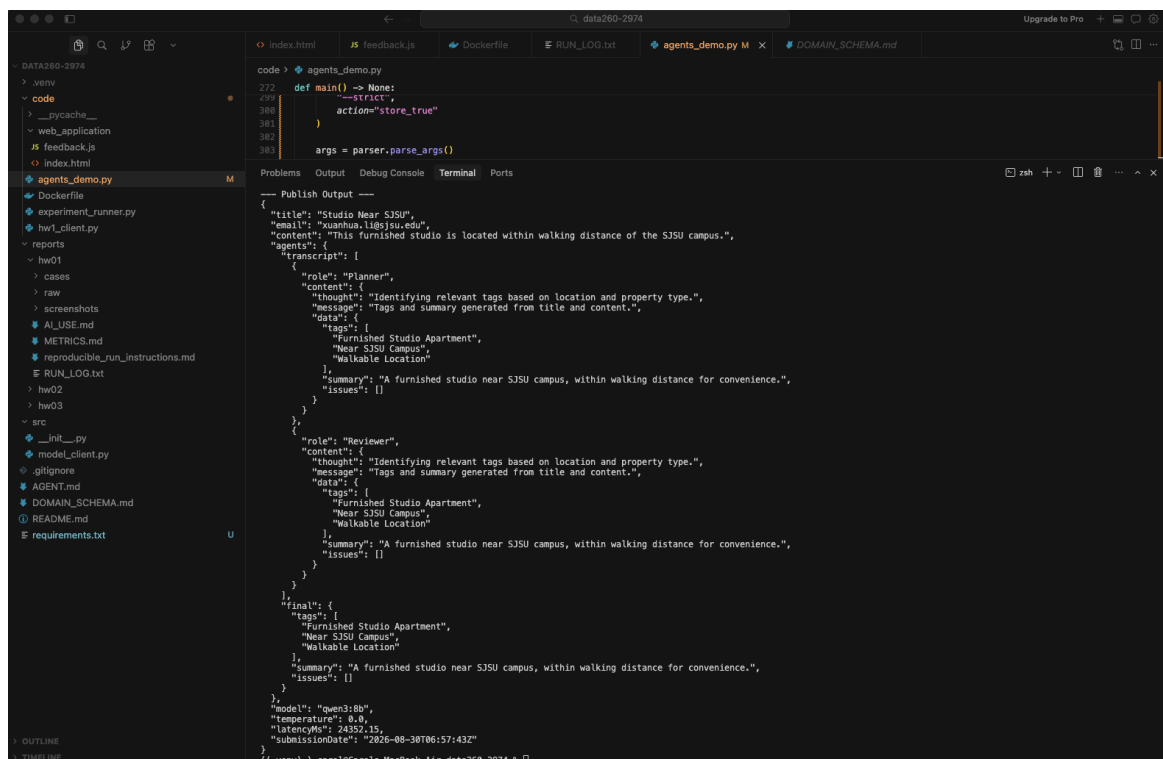
```
code > agents_demo.py
277 def main() -> None:
278     """
279     """
280     action="store_true"
281     parser.add_argument(
282         "--strict",
283         action="store_true"
284     )
285     args = parser.parse_args()

Problems Output Debug Console Terminal Ports
((.venv) ) carol@Carols-MacBook-Air data268-2974 % python -m py_compile code/agents_demo.py
((.venv) ) carol@Carols-MacBook-Air data268-2974 % python code/agents_demo.py --title "Studio Near SJSU" --content "This furnished studio is located within walking distance of the SJSU campus." --email "xuanhua.li@sjsu.edu" --model "qwen3:8b" --temperature 0.0 --strict

--- Planner Output ---
{
  "thought": "Identifying relevant tags based on location and property type.",
  "message": "Tags and summary generated from title and content.",
  "data": {
    "tags": [
      "Furnished Studio Apartment",
      "Near SJSU Campus",
      "Walkable Location"
    ],
    "summary": "A furnished studio near SJSU campus, within walking distance for convenience.",
    "issues": []
  }
}

--- Reviewer Output ---
{
  "thought": "Identifying relevant tags based on location and property type.",
  "message": "Tags and summary generated from title and content.",
  "data": {
    "tags": [
      "Furnished Studio Apartment",
      "Near SJSU Campus",
      "Walkable Location"
    ],
    "summary": "A furnished studio near SJSU campus, within walking distance for convenience.",
    "issues": []
  }
}

--- Finalized Output ---
{
  "thought": "Identifying relevant tags based on location and property type.",
  "message": "Tags and summary generated from title and content.",
  "data": {
    "tags": [
      "Furnished Studio Apartment",
      "Near SJSU Campus",
      "Walkable Location"
    ],
    "summary": "A furnished studio near SJSU campus, within walking distance for convenience.",
    "issues": []
  }
}
```



```
code > agents_demo.py
277 def main() -> None:
278     """
279     """
280     action="store_true"
281     parser.add_argument(
282         "--strict",
283         action="store_true"
284     )
285     args = parser.parse_args()

Problems Output Debug Console Terminal Ports
--- Publish Output ---
{
  "title": "Studio Near SJSU",
  "email": "xuanhua.li@sjsu.edu",
  "content": "This furnished studio is located within walking distance of the SJSU campus.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": {
          "thought": "Identifying relevant tags based on location and property type.",
          "message": "Tags and summary generated from title and content.",
          "data": {
            "tags": [
              "Furnished Studio Apartment",
              "Near SJSU Campus",
              "Walkable Location"
            ],
            "summary": "A furnished studio near SJSU campus, within walking distance for convenience.",
            "issues": []
          }
        }
      },
      {
        "role": "Reviewer",
        "content": {
          "thought": "Identifying relevant tags based on location and property type.",
          "message": "Tags and summary generated from title and content.",
          "data": {
            "tags": [
              "Furnished Studio Apartment",
              "Near SJSU Campus",
              "Walkable Location"
            ],
            "summary": "A furnished studio near SJSU campus, within walking distance for convenience.",
            "issues": []
          }
        }
      }
    ],
    "final": {
      "tags": [
        "Furnished Studio Apartment",
        "Near SJSU Campus",
        "Walkable Location"
      ],
      "summary": "A furnished studio near SJSU campus, within walking distance for convenience.",
      "issues": []
    }
  },
  "model": "qwen3:8b",
  "temperature": 0.0,
  "latencyMs": 24352.15,
  "submissionDate": "2026-08-30T06:57:43Z"
}

((.venv) ) carol@Carols-MacBook-Air data268-2974 %
```

The local qwen3:8b model produced valid structured JSON for the Planner, Reviewer, and Finalizer stages. The Publish output combined the original submission, agent transcript, final result, model settings, latency, and timestamp.

3. Short answers:

Q1: The three final tags

"Furnished Studio Apartment",
"Near SJSU Campus",
"Walkable Location"

Q2: The final summary (≤ 25 words)

"summary": "A furnished studio near SJSU campus, within walking distance for convenience."

Q3: Did the Reviewer agent change anything? (yes/no + explanation)

No. The Planner already produced exactly three distinct topical tags and a summary under 25 words, so the Reviewer reported no issues.

4. An explanation of each step in your own words.

Planner:

The Planner receives the fixed rental-listing title and content. It creates an initial structured JSON response containing exactly three tags, a short summary, and an issues list.

Reviewer:

The Reviewer receives both the original rental-listing input and the Planner output. It checks whether the tags are distinct and relevant and whether the summary contains no more than 25 words. For this input, it found no problems, so it kept the Planner result unchanged.

Finalizer:

The Finalizer compares the Planner and Reviewer outputs and produces the final validated result. The Python program then verifies that there are exactly three non-empty and distinct tags and that the summary is within the word limit.

Publish:

The Publish step packages the original title, email, and content together with the agent transcript, final tags and summary, model name, temperature, latency, and submission timestamp.

Measuring Non-Determinism (Part 3)

Before running this experiment, pick one fixed domain-relevant input (a title and content) and save it once, unchanged, as `reports/hw01/cases/nondeterminism_input.json`.

```
reports > hw01 > cases > {} nondeterminism_input.json > ...  
1 {  
2   "title": "Studio Near SJSU",  
3   "content": "This furnished studio is located within walking distance of the SJSU campus."  
4 }
```

1. Run your `agents_demo.py` pipeline on that exact input 20 times at temperature 0.7, and 20 times at temperature 0.0 (40 runs total).

I used the same fixed rental-listing input for every experiment:

- Title: Studio Near SJSU
- Content: This furnished studio is located within walking distance of the SJSU campus.
- Model: qwen3:8b
- Runs at temperature 0.0: 20
- Runs at temperature 0.7: 20
- Total successful runs: 40

The fixed input was saved in:

`reports/hw01/cases/nondeterminism_input.json`

I ran the experiment with the following command:

```
python code/experiment_runner.py --email "xuanhua.li@sjsu.edu" --model "qwen3:8b" --runs 20
```

The experiment runner submitted the same input 20 times at each temperature. Failed or invalid JSON responses would be retried and would not count as successful runs.

```

--- Experiment Metrics ---
{
  "0.0": {
    "runCount": 20,
    "distinctTagSets": 1,
    "tagsInAllRuns": [
      "Furnished Studio Apartment",
      "Near SJSU Campus",
      "Walkable Location"
    ],
    "tagsInExactlyOneRun": [],
    "latencyMs": {
      "p50": 18078.26,
      "p95": 19686.47,
      "p99": 19889.03
    }
  },
  "0.7": {
    "runCount": 20,
    "distinctTagSets": 10,
    "tagsInAllRuns": [],
    "tagsInExactlyOneRun": [
      "Furnished Studio Living",
      "SJSU Campus Proximity",
      "SJSU Proximity",
      "Walkable Campus",
      "Walking Distance Living",
      "sjsu campus nearby",
      "walkable studio",
      "walking distance from SJSU",
      "walking distance housing",
      "walking distance location",
      "walking distance to SJSU"
    ],
    "latencyMs": {
      "p50": 27728.17,
      "p95": 31670.03,
      "p99": 32451.89
    }
  }
}

Saved raw results to:
/Users/carol/Documents/data260-2974/reports/hw01/raw/nondeterminism_runs.json
((.venv) ) carol@Carols-MacBook-Air data260-2974 % 

```

2. For each temperature, report:
 - the number of distinct tag sets produced
 - the tags that appeared in all 20 runs
 - the tags that appeared in exactly one run
 - latency p50 / p95 / p99

The final non-determinism experiment produced the following results:

Metric	Temperature 0.0	Temperature 0.7
Successful runs	20	20

Distinct tag sets	1	10
p50 latency (ms)	18,078.26	27,728.17
p95 latency (ms)	19,686.47	31,670.03
p99 latency (ms)	19,889.03	32,451.89

At temperature 0.0, the following tags appeared in all 20 runs:

- Furnished Studio Apartment
- Near SJSU Campus
- Walkable Location

No tag appeared in exactly one temperature-0.0 run.

At temperature 0.7, no tag appeared in all 20 runs. The following tags appeared in exactly one run:

- Furnished Studio Living
- SJSU Campus Proximity
- SJSU Proximity
- Walkable Campus
- Walking Distance Living
- sjsu campus nearby
- walkable studio
- walking distance from SJSU
- walking distance housing
- walking distance location
- walking distance to SJSU

3. Save the raw per-run results (tags + latency, all 40 runs) as JSON/CSV in reports/hw01/raw/.

The complete raw experiment results were saved in:

reports/hw01/raw/nondeterminism_runs.json

For each successful run, the file records the temperature, run number, number of attempts, generated tags, generated summary, latency in milliseconds, and timestamp. It also contains the aggregated metrics for both temperatures.

4. Using your own numbers, explain what two users sending identical input might see, and

give one example where run-to-run variation is acceptable and one where it is not.

At temperature 0.0, the model generated the same set of three tags in all 20 runs. This indicates highly consistent behavior for the fixed input under this configuration.

At temperature 0.7, the model produced 10 distinct tag sets across 20 runs. The tags generally described the same rental property, but their wording varied. For example, the model used related expressions such as “Near SJSU Campus,” “SJSU Proximity,” and “walking distance to SJSU.”

Therefore, two users who provide identical input may receive different but semantically related outputs when the temperature is higher. This variation is acceptable for creative or descriptive fields, such as search tags, short summaries, and marketing language.

The same variation would not be acceptable for factual or legally significant information, such as the property address, rental price, lease terms, eligibility requirements, or safety information. Those fields should be preserved or validated rather than generated freely by the model.

The temperature-0.7 runs were also slower in this experiment. Their median latency was 27,728.17 ms, compared with 18,078.26 ms at temperature 0.0. This result describes this particular local experiment and does not prove that temperature alone always causes higher latency.

Model Client and Token Accounting (Part 4)

1. Create a reusable model-adapter module at `src/model_client.py`, defining a stable interface such as `complete(messages, tools=None)`. All model calls must go through this Adapter.

The reusable model adapter is located at the required path:

`src/model_client.py`

The `ModelClient` class provides a stable `complete(messages, tools=None)` interface. Application programs send messages through this interface instead of calling `ChatOllama` directly.

The adapter is responsible for:

- Configuring the Ollama model and generation settings
- Converting application messages into the format expected by the model
- Sending the request to the local `qwen3:8b` model
- Returning the generated text and token-usage information
- Keeping model-specific implementation details separate from the application code

Both `code/agents_demo.py` and `code/hw1_client.py` use this shared adapter. This design makes it possible to change the underlying model implementation later without rewriting every application that uses it.

```
model_client.py

class ModelClient:
    """Reusable adapter for local Ollama model calls."""

    def __init__(
        self,
        model_name: str = "qwen3:8b",
        temperature: float = 0.0,
        output_format: str | dict[str, Any] | None = None,
        reasoning: bool = False,
        base_url: str = "http://localhost:11434"
    ) -> None:
        model_options: dict[str, Any] = {
            "model": model_name,
            "temperature": temperature,
            "reasoning": reasoning,
            "base_url": base_url,
            "num_ctx": 2048,
            "num_predict": 1024
        }

        if output_format is not None:
            model_options["format"] = output_format

        self._model = ChatOllama(**model_options)
```

```

src > model_client.py
19 class ModelClient:
29     ) -> None:
43
44     def complete(
45         self,
46         messages: list[dict[str, str]],
47         tools: list[Any] | None = None
48     ) -> CompletionResult:
49         """
50         Send messages to the model through one stable interface.
51
52         The optional tools parameter is included for future homework.
53         """
54         model = self._model
55
56         if tools:
57             model = self._model.bind_tools(tools)
58
59         response = model.invoke(messages)
60
61         if isinstance(response.content, str):
62             content = response.content
63         else:
64             content = json.dumps(
65                 response.content,
66                 ensure_ascii=False
67             )
68
69         usage = response.usage_metadata or {}
70         metadata = response.response_metadata or {}
71
72         input_tokens = int(
73             usage.get(
74                 "input_tokens",
75                 metadata.get("prompt_eval_count", 0)
76             )
77         )

```

```

69         usage = response.usage_metadata or {}
70         metadata = response.response_metadata or {}
71
72         input_tokens = int(
73             usage.get(
74                 "input_tokens",
75                 metadata.get("prompt_eval_count", 0)
76             )
77         )
78
79         output_tokens = int(
80             usage.get(
81                 "output_tokens",
82                 metadata.get("eval_count", 0)
83             )
84         )
85
86         total_tokens = int(
87             usage.get(
88                 "total_tokens",
89                 input_tokens + output_tokens
90             )
91         )
92
93         return CompletionResult(
94             content=content,
95             input_tokens=input_tokens,
96             output_tokens=output_tokens,
97             total_tokens=total_tokens,
98             metadata=metadata
99         )

```

2. Create `hw1_client.py` as a small command-line demo that imports the adapter.

I implemented the interactive code-review client in:

`code/hw1_client.py`

The client loads the instructions from `AGENT.md` and sends them to the model as the system message. It accepts multiple user prompts in one conversation and stores the conversation history so that earlier turns remain available to the model.

3. Create `AGENT.md` asking for strict bullet-only code review, and verify the model actually follows it.

```
AGENT.md > [abc] # Code Review Agent Instructions
1  # Code Review Agent Instructions
2
3  - Act as a strict code reviewer.
4  - Respond only with bullet points.
5  - Begin every non-empty response line with "- ".
6  - Review code for correctness, clarity, security, and maintainability.
7  - Identify specific problems and suggest concise improvements.
8  - Do not use headings.
9  - Do not use paragraphs.
10 - Do not use numbered lists.
11 - Do not use Markdown code fences.
12 - Do not include any text before or after the bullet list.
13 - Every response must contain at least one bullet point.
14 - If no problems are found, respond with "- No issues found."
15 - Review only the exact code provided by the user.
16 - Do not invent operators, variables, syntax, or behavior that is not present in the submitted code.
17 - If the submitted code has no identifiable problems, respond with "- No issues found."
```

4. After every model response, print input tokens, output tokens, and total tokens for that turn. On exit, print cumulative input tokens, output tokens, and turn count.

5. Add a `/stats` command showing turn count, cumulative token counts, and serialized conversation-history length, without altering the history.

6. Run a five-turn conversation; record `/stats` output after turn 3 and after turn 5.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw1_client.py --model "qwen3:8b"
HW1 Code Review Client
Commands: /stats, /exit
Model: qwen3:8b
```

You> Review only this exact Python code for correctness. Do not invent any code: def divide(a, b): return a / b

Model>

- The function name 'divide' is misleading as it performs division by two values, not just one.
- The code is missing error handling for division by zero.
- The function does not handle cases where 'b' is zero, which will raise a 'ZeroDivisionError'.
- The function does not handle non-numeric inputs, which will raise a 'TypeError'.
- The function lacks a docstring for clarity and documentation.

Token usage:

Input tokens: 214
Output tokens: 91
Total tokens: 305
Bullet-only check: PASS

You> Review only this exact Python code for correctness. Do not invent any code: def first_item(items): return items[0]

Model>

- The function assumes 'items' is a sequence with at least one element, but does not handle empty input, which will raise an 'IndexError'.
- The function does not handle non-numeric elements in 'items', which will raise a 'TypeError'.
- The function lacks a docstring for clarity and documentation.
- The function name 'first_item' is misleading as it performs an operation (division) rather than simply returning an item.

Token usage:

Input tokens: 339
Output tokens: 93
Total tokens: 432
Bullet-only check: PASS

You> Review only this exact Python code for security. Do not invent any code: password = "admin123"

Model>

- The password is hardcoded in plain text, which poses a security risk if the code is exposed.
- There is no input validation or sanitization for the password.
- The password is not stored securely (e.g., using hashing or encryption).
- The password is not protected against brute-force attacks.

Token usage:

Input tokens: 464
Output tokens: 64
Total tokens: 528
Bullet-only check: PASS

You> /stats

```
--- Statistics ---
Turn count: 3
Cumulative input tokens: 1017
Cumulative output tokens: 248
Cumulative total tokens: 1265
Serialized history length: 1673
```

You> []

You> Review only this exact Python code for maintainability. Do not invent any code: def calculate(x): return x * 0.0825

Model>

- The function lacks a docstring, making it unclear what the calculation represents.
- The magic number '0.0825' is not explained, reducing readability and maintainability.
- The function name 'calculate' is too generic and does not indicate the purpose of the calculation.
- There is no error handling for non-numeric inputs, which will raise a 'TypeError'.

Token usage:

Input tokens: 567
Output tokens: 81
Total tokens: 648
Bullet-only check: PASS

You> Review only this exact Python code for correctness. Do not invent any code: def average(values): return sum(values) / len(values)

Model>

- The line '/no_think' is not valid Python syntax and will cause a 'SyntaxError'.

Token usage:

Input tokens: 684
Output tokens: 22
Total tokens: 706
Bullet-only check: PASS

You> /stats

```
--- Statistics ---
Turn count: 5
Cumulative input tokens: 2268
Cumulative output tokens: 351
Cumulative total tokens: 2619
Serialized history length: 2507
```

You> []

Token Accounting Results

The five-turn conversation produced the following token counts:

Turn	Input Tokens	Output Tokens	Total Tokens	Cumulative Total	Bullet-Only Check
1	214	91	305	305	PASS
2	339	93	432	737	PASS
3	464	64	528	1,265	PASS
4	567	81	648	1,913	PASS
5	684	22	706	2,619	PASS

The complete per-turn data is stored in: `reports/hw01/raw/token_counts.json`

Statistics After Turn 3

Metric	Value
Turn count	3
Cumulative input tokens	1,017
Cumulative output tokens	248
Cumulative total tokens	1,265
Serialized history length	1,673

Statistics After Turn 5

Metric	Value
Turn count	5
Cumulative input tokens	2,268
Cumulative output tokens	351
Cumulative total tokens	2,619
Serialized history length	2,507

The input-token count increased during the conversation because the client resent the system instructions and relevant conversation history with every new request.

7. In your Report and README, answer: why is prior conversation context resent with every turn? How is a system prompt different from a user message? Why do input tokens grow over a conversation? What eventually limits that growth?

- Why is prior conversation context resent with every turn?

A model request is stateless: the model does not automatically remember earlier API calls. To continue a conversation, the client sends the relevant previous user and assistant messages again with each new request. This allows the model to interpret the current message using earlier context.

- How is a system prompt different from a user message?

A system prompt defines the model's overall role, behavior, and response constraints. In this project, the contents of `AGENT.md` are sent as the system message and require bullet-only code reviews.

A user message contains the current task or question. It has a lower instructional role and should operate within the behavior established by the system prompt.

- Why do input tokens grow over a conversation?

Input tokens grow because each new request contains the system instructions, previous conversation history, and the newest user message. As additional turns are added to the history, more text must be serialized and sent to the model.

In the recorded run, the input-token count increased from 214 tokens in turn 1 to 684 tokens in turn 5.

- What eventually limits that growth?

The model's context window limits how many tokens can be processed in one request. Once the combined system prompt, conversation history, and current message approach that limit, the application must remove older messages, summarize earlier context, retrieve only relevant history, or use a model with a larger context window.

Longer context also increases latency and resource usage, so practical limits may be reached before the maximum context-window size.

AI Use

1. What I Used an AI Assistant For and What I Did Myself

I used an AI coding assistant throughout the development process for requirement interpretation, implementation support, code explanations, debugging assistance, and documentation feedback.

My work involved adapting and integrating the project code for my assigned rental-housing domain and personal configuration. I connected the HTML form and JavaScript behavior, configured and tested the Docker application, set up the local Ollama environment, integrated the agent pipeline and reusable model client, configured the experiment runner, and organized the required source code and report artifacts.

I reviewed and modified the implementation during testing. I diagnosed empty model responses, adjusted the reasoning and output settings, strengthened the agent instructions, verified the browser and console behavior, ran the 40-run experiment, examined incorrect model responses, and recorded the resulting logs, token counts, metrics, and screenshots.

I was responsible for testing the completed system against the homework requirements and deciding which changes and results were included in the final submission.

2. One AI-Produced Output That Was Wrong or Unsuitable

The AI assistant suggested enabling reasoning mode in the interactive model client to improve the quality of the model's code reviews. This suggestion was unsuitable for my local configuration. With reasoning enabled, the model consumed the complete output allowance but produced no visible final response. Increasing the output limit from 400 to 1024 tokens did not solve the problem.

3. How I Detected or Verified the Problem

I did not assume that the suggested setting was correct. I ran the interactive client and tested it with multiple code-review prompts. The terminal showed an empty `Model>` section, an output-token count exactly equal to the configured limit, and `Bullet-only check: FAIL`.

I repeated the test after increasing the output limit from 400 to 1024 tokens. The same problem occurred, which confirmed that simply increasing the limit was not an effective solution.

4. What I Changed and Why It Works Now

I changed the interactive client from `reasoning=True` to `reasoning=False`. I then restarted the client and tested it again. The model produced a visible bullet-point response, and the automatic bullet-only check passed.

I used the revised configuration for the formal five-turn conversation. All five turns produced visible responses and passed the required format check. Disabling reasoning worked because the model no longer spent its entire output allowance on hidden reasoning before producing the final response.

I still reviewed the visible responses manually because passing the format check did not guarantee that every code-review statement was factually correct.

Reproducibility

The complete reproduction instructions are available at:

reports/hw01/reproducible_run_instructions.md

The project requires Python 3.12, Docker Desktop, Ollama, and the qwen3:8b model. Python dependencies are recorded in requirements.txt.

The documented workflow includes:

1. Creating and activating the Python virtual environment
2. Installing the required Python packages
3. Downloading and testing qwen3:8b with Ollama
4. Running the Planner–Reviewer–Finalizer pipeline
5. Running the 40-run non-determinism experiment
6. Running the five-turn interactive model client
7. Building and running the Docker web application
8. Running the Homework 1 verification program

The raw results, metrics, execution log, screenshots, token data, and verification output are stored under reports/hw01/.

Homework Verification

The automated verification result is stored in reports/hw01/verification.json.

Verification check	Result
Required files present	PASS
40-run non-determinism experiment	PASS
Five-turn token accounting	PASS

Shared model adapter usage	PASS
AGENT.md bullet-only instructions	PASS
Docker web application HTTP test	PASS
Overall verification	PASS